Projet Graphes Gene Ontology – Homo sapiens

Laroussi Bachri

2025-12-22

Contents

1 Contexte **2**

2 Analyse **2**

2.1 Ressources disponibles 2

2.2 Besoins identifiés 3

3 Conception **4**

3.1 Représentation choisie 4

3.2 Algorithmes 5

3.3 Réalisation 7

4 Résultats **9**

4.1 Résultats exploratoires. 9

4.2 Répartition des GOTerms par ontologie. 9

4.3 Profondeur maximale par ontologie 10

4.4 Distribution du nombre de GeneProducts par GOTerm. 11

4.5 Distribution du nombre d'annotations par GeneProduct 12

4.6 Répartition des evidence codes. 12

5 Discussion **13**

6 Bilan et perspective **14**

7 Section spéciale méthodologie de travail **14**

8 Annexes **15**

1 Contexte

La Gene Ontology (GO) constitue aujourd'hui l'un des outils les plus utilisés en annotation fonctionnelle des gènes et des protéines. Elle organise les connaissances biologiques sous forme d'un graphe orienté acyclique (DAG) structuré en trois branches principales : Biological Process, Molecular Function et Cellular Component. Chaque GeneProduct, peut être associé à un ou plusieurs GOTerms décrivant sa fonction, sa localisation cellulaire ou les processus auxquels il participe.

L'objectif du projet est de développer une bibliothèque Python permettant de manipuler efficacement la Gene Ontology et les annotations associées. Cette bibliothèque doit s'appuyer sur un module générique de manipulation de graphes (fonctions de traitement et parcours des graphes) , charger le graphe de la Gene Ontology , charger les annotations (format GOA/GAF) , les analyser et calculer des statistiques sur la profondeur du graphe et la distribution des annotations.

Pour ce projet, l'organisme choisi est Homo sapiens, dont les annotations GO sont riches et bien maintenues.

2 Analyse

La mise en place d'outils pour manipuler la Gene Ontology nécessite d'identifier d'une part les ressources disponibles, et d'autre part les fonctionnalités manquantes qu'il faut concevoir et implémenter.

2.1 Ressources disponibles

2.1.1 Données

Deux types de fichiers standards permettent de travailler avec la GO :

Fichier OBO (Ontology File) Contient la définition complète des GOTerms : identifiants, noms, namespace, définitions, relations hiérarchiques (is_a, part_of), ainsi que les identifiants alternatifs.
Permet de reconstruire la structure du graphe orienté acyclique. Le fichier OBO utilisé est disponible ici: https://current.geneontology.org/ontology/go-basic.obo
Il a été publié le 10-10-2025 et sa version est 1.2.

Fichiers GOA (Gene Ontology Annotation) Fournissent les associations entre GeneProducts et GOTerms. Contient des méta-informations : nom du produit, synonymes, evidence codes, date d'assignation...
Le fichier avec lequel nous avons travaillé a été générée le 10-11-2025 par Uniprot et sa version est 2.2 et le lien de téléchargement est disponible ici : https://ftp.ebi.ac.uk/pub/databases/GO/goa/proteomes/25.H_sapiens.goa

Le fichier a ete mis à jour le 2-12-2025.

2.1.2 Organisation du code:

Le projet est structuré selon une organisation modulaire classique en Python, distinguant clairement le code source, les données et les tests. Les modules principaux (graphmaster.py et geneontology.py) sont regroupés dans un répertoire dédié, tandis que les tests unitaires sont placés dans un répertoire séparé (tests).

Afin de garantir une reproductibilité conforme aux bonnes pratiques Python, les tests sont exécutés en tant que modules (python -m) depuis la racine du projet. Cette méthode permet à l'interpréteur Python de résoudre correctement les imports entre modules sans modifier manuellement le chemin d'exécution (PYTHONPATH). Elle assure également une meilleure portabilité du code et facilite l'intégration ultérieure de tests automatisés.

Le projet repose sur deux modules :

graphmaster.py - Module générique de manipulation de graphes (dictionnaires de sommets et d'arêtes). Fournit des fonctions de base : ajout de sommets, ajout d'arcs, neighbors, BFS, DFS, détection de cycles, tri topologique, sous-graphe induit.

geneontology.py Ce module contient le code minimal pour charger la GO (load_OBO) et les annotations (load_GOA) , et des fonctions pour obtention de GOTerms à partir d'un GeneProduct et inversement ainsi qu'une fonction pour calculer profondeur maximale dans la GeneOntology sont à compléter.

2.2 Besoins identifiés

Le projet nécessite un ensemble clair de besoins fonctionnels :

a) **Représentation interne de la GO**

Il est nécessaire de construire un graphe orienté permettant de représenter les relations hiérarchiques is_a et part_of (en particulier leur direction : enfant → parent) et les liens GeneProduct → GO term créés lors du chargement du GAF.

Ce graphe doit permettre une exploration rapide et efficace de la structure.

b) **Navigation et interrogation du DAG**

Plusieurs opérations doivent être rendues possibles.

- Obtenir les parents et les enfants d'un terme GO,
- Parcourir l'ontologie dans les deux directions (ancêtres et descendants),
- Récupérer les GOTerms associés à un GeneProduct (directs ou incluant les ancêtres),

- Obtenir les GeneProducts associés à un terme GO (directs ou via les descendants).

c) **Analyse structurelle de la GO**

Le projet demande le calcul du chemin le plus long dans le DAG (profondeur maximale), l'identification du terme le plus profond dans chacune des trois ontologies et la reconstruction du chemin complet entre termes spécifiques et généraux.

Cela nécessite un algorithme dédié basé sur un tri topologique et de la programmation dynamique.

d) **Calcul de statistiques biologiques**

- Nombre de sommets et d'arcs.
- répartition par ontologie (BP, MF, CC),
- nombre de GeneProducts annotés,
- distribution du nombre de GP par GOTerm.
- comptage du nombre total de GOTerms et GeneProducts
- nombre total d'annotations

e) **Validations et tests**

Le code doit être : - robuste, - testé unitairement, - capable de gérer des fichiers de grande taille.

Pour cela, nous allons creer des fichiers qui testent notre module. Ces tests ont été efféctués avec un mini graphe construit sur papier pour verifier le bon fonctionnement des fonctions.

Nous avons crée 2 script Python permettant de tester nos modules graphmaster et geneontology.

Les fichiers portent respectivement les noms des modules testés.

Un fichier test sur le graphe de la Gene Ontology et les annotations de Homo Sapiens est générée pour verifier le bon fonctionnement des fonctions, nous avons cherché les informations manuellement.

3 Conception

3.1 Représentation choisie

Pour ce projet, nous avons choisi de conserver une représentation du graphe sous forme de dictionnaires, implémentées avec Python :

- g['nodes'] : dictionnaire id → attributs,

- g['edges'] : dictionnaire u → {v → attributs} représentant les arcs sortants de u.

Ce choix est celui qui est utilisé par les fonctions fournies en TP **load_GOA** , **load_OBO**

Ce choix permet :

- un accès rapide aux voisins d'un sommet (O(degré)),
- une bonne scalabilité pour des graphes clairsemés comme la Gene Ontology,
- une intégration directe avec le module générique graphmaster.py construit en TP.
- **GOTerms (GOTerm)** : sommets avec l'attribut **type = 'GOTerm'** et un namespace parmi biological_process, molecular_function, cellular_component.
- **GeneProduct** : Sommets avec type = 'GeneProduct', et des attributs comme id, name, aliases.
- **Arcs GO –> GO** : Les relations **is_a** et **part_of** sont représentées par des arcs orientés de l'enfant vers le parent : enfant → parent Cela correspond à une lecture "terme spécifique → terme plus général".
- **Arcs d'annotation GP→GO** : Les associations entre GeneProducts et GOTerms sont représentées par des arcs orientés GeneProduct → GOTerm avec un attribut **relationship = 'annotation'**.

3.2 Algorithmes

3.2.1 GeneProducts associés à un GOTerm (directs et descendants)

L'objectif de cette fonction est de déterminer l'ensemble des GeneProducts associés à un terme GO donné, soit par annotation directe, soit indirectement via ses termes descendants reliés par les relations is_a et part_of.

L'algorithme commence par vérifier que l'identifiant fourni correspond bien à un terme GO existant dans le graphe. En mode non récursif, il retourne directement les GeneProducts annotés au terme considéré à partir de l'index, ce qui garantit un accès immédiat.

Lorsque l'option récursive est activée, l'algorithme exploite la structure hiérarchique de la Gene Ontology en parcourant les termes descendants du terme GO d'intérêt. Pour chacun de ces termes, les GeneProducts correspondants sont récupérés via l'index et agrégés dans un ensemble unique. Cette approche est biologiquement justifiée, car une annotation portée par un terme GO spécifique implique une annotation implicite à ses termes plus généraux, et permet en pratique d'associer correctement les GeneProducts à un concept GO donné tout en conservant de bonnes performances.

3.2.2 GOTerms associés à un GeneProduct (directs et ancêtres)

L'objectif de cette fonction est de déterminer l'ensemble des GOTerms associés à un GeneProduct donné, identifié par gp_id. Cette association peut être limitée aux annotations explicites fournies par les fichiers GOA, ou étendue aux termes plus généraux de la Gene Ontology lorsque l'option récursive est activée.

Dans un premier temps, l'algorithme vérifie que le GeneProduct considéré existe dans le graphe. En cas d'absence, un ensemble vide est retourné. Lorsque l'option récursive n'est pas choisie, la fonction se limite à la récupération des GOTerms directement annotés au GeneProduct, correspondant aux relations de type annotation. Il suffit de regarder les voisins de notre GeneProduct d'intêret et de verifier si c'est bien un GOTerm.

Lorsque l'option récursive est choisie, l'algorithme exploite la structure hiérarchique de la Gene Ontology en traitant les termes plus généraux. À partir des GOTerms annotés explicitement, un parcours en largeur est effectué vers les termes parents, en suivant les relations hiérarchiques is_a et part_of. Cette approche permet d'inclure l'ensemble des GOTerms plus généraux associés implicitement au GeneProduct. Les termes visités sont mémorisés afin d'éviter les redondances et de garantir l'unicité des résultats.

3.2.3 Calcul de la profondeur maximale des ontologies GO

L'objectif de cet algorithme est de déterminer la profondeur maximale de la hiérarchie de la Gene Ontology (GO). La profondeur est définie comme la longueur du plus long chemin hiérarchique reliant un terme GO spécifique à un terme GO plus général, en suivant exclusivement les relations hiérarchiques is a et part of.

Afin de calculer cette profondeur séparément pour chacune des ontologies (Biological Process, Molecular Function et Cellular Component), des fonctions auxiliaires sont utilisées pour sélectionner les sommets appartenant à une ontologie donnée à partir de leur attribut namespace. Ces sommets sont ensuite utilisés pour construire un sous-graphe induit contenant uniquement les GOTerms sélectionnés ainsi que les relations hiérarchiques les reliant.

La hiérarchie de la Gene Ontology étant définie comme un graphe orienté acyclique (DAG), cette propriété est exploitée par l'algorithme. Les arcs du graphe sont orientés du terme le plus spécifique vers le terme le plus général (enfant $\rightarrow$ parent), conformément aux relations is_a et part_of.

L'algorithme débute par un tri topologique des sommets du sous-graphe induit, obtenu à partir d'un parcours en profondeur (DFS). Ce tri fournit un ordre linéaire compatible avec la structure acyclique du graphe. Afin de calculer correctement la profondeur des termes, les sommets sont ensuite parcourus dans l'ordre topologique inverse, ce qui garantit que les termes parents ont déjà été traités lors du calcul de la profondeur de leurs termes enfants.

Une profondeur initiale égale à zéro est associée à chaque terme GO. Pour chaque terme, l'algorithme examine ses voisins sortants, qui correspondent à ses termes parents dans la hiérarchie. La profondeur du terme courant est alors mise à jour en prenant la valeur maximale entre sa profondeur actuelle et la

profondeur de son parent augmentée d'une unité. Ce mécanisme correspond à un calcul dynamique du plus long chemin dans un DAG.

Le chemin hiérarchique correspondant est alors reconstruit en remontant les prédécesseurs depuis ce terme jusqu'à la racine de la hiérarchie. Pour des raisons de lisibilité, ce chemin peut être présenté soit dans le sens spécifique → général (feuille vers racine), soit dans le sens inverse.

3.3 Réalisation

3.3.1 Temps d'exécution et complexité

Analyse de la Complexité de Geneproducts()

Dans une première implémentation naïve (presence dans geneontology.py), la récupération des GeneProducts associés à un terme GO (en mode récursif) reposait sur des parcours répétés du graphe afin d'identifier, pour chaque descendant, les annotations correspondantes, ce qui pouvait conduire à une complexité élevée.

Dans la version finale, un index inversé est construit (association GOTerm → ensemble de GeneProducts annotés) ainsi qu'un index hiérarchique (association parent → ensemble des enfants). L'exploration des descendants se fait alors par un parcours en largeur (BFS) sur l'index hiérarchique, puis une concatenation des ensembles via l'index inversé. Cette approche ramène le coût principal à un parcours linéaire du sous-graphe exploré qui est beaucoup plus optimisé que la première implémentation.

Analyse de la Complexité de GOTerms()

Dans le cas non récursif, la complexité de l'algorithme est dominée par la récupération des annotations directes, qui dépend du nombre d'arêtes sortantes du GeneProduct et peut être considérée comme linéaire en fonction du nombre d'annotations explicites.

Dans le cas récursif, l'algorithme réalise un parcours en largeur de la hiérarchie GO à partir des termes annotés directement. La complexité de ce parcours est en O(V_GO + E_GO), où V_GO correspond au nombre de GOTerms visités et E_GO au nombre de relations hiérarchiques parcourues.

Analyse de la Complexité de maxdepth_go()

Le calcul de la profondeur repose sur un tri topologique et un parcours linéaire des sommets et des arcs du graphe. Sa complexité temporelle est donc linéaire en le nombre de sommets et d'arcs du sous-graphe considéré. La complexité de ce parcours est également O(V_GO + E_GO).

Des algorithmes comme Dijkstra, Bellman-Ford ou Floyd-Warshall ont été écartés car ils sont conçus pour des graphes pondérés et auraient une complexité inutilement élevée (notamment Floyd Warshall) dans notre cas.

3.3.2 Problèmes rencontrés liés a la complexité

Fonction GeneProducts()

Dans un premier essai d'implémentation, on obtenait les resultats attendus mais cela prenait un temps trop important. Afin d'améliorer l'efficacité des requêtes, un index inversé est construit, associant à chaque terme GO l'ensemble des GeneProducts qui lui sont directement annotés. Ce choix permet d'éviter des parcours répétés de l'ensemble du graphe lors de la récupération des annotations. L'utilisation de cet index est activée par défaut dans la fonction.

La première implementation dépassait les 800 secondes avant interruption.

Nous allons analyser rapidement le temps d'exécution de nos programmes par notre machine.

Les temps sont satisfaisants et nettement améliorés.

Table 1: Résultats des performances (temps)

Chargement du graphe Gene Ontology.	0.51s
Chargement des annotations Gene Ontology (Homo sapiens)	2.24s
Obtention du nombre de GeneProducts annotés à 'GO:0008150'	0.32s
Obtention du nombre de GeneProducts annotés à 'GO:0008150' (en recursive)	0.28s
Obtention du nombre de GOTerm annotés à 'P04637'	0.00012s
Obtention du nombre de GOTerm annotés à 'P04637' (en recursive)	0.00050s
Plus long chemin entre GOTerms pour biological_process	0.40s
Plus long chemin entre GOTerms pour molecular_function	0.11s
Plus long chemin entre GOTerms pour cellular_component	0.05s

4 Résultats

4.1 Résultats exploratoires.

Après chargement de la Gene Ontology et des annotations associées à Homo sapiens, le graphe obtenu contient 105 045 sommets et 607 727 arêtes. Les sommets correspondent aux GOTerms (toutes ontologies confondues) et aux GeneProducts annotés pour l'organisme étudié. Les arêtes représentent les relations hiérarchiques entre GOTerms (is_a, part_of) ainsi que les relations d'annotation reliant les GeneProducts aux GOTerms.

Table 2: Figure 1 : Principaux résultats du graphe GO/annotations.

Nombre de sommets	105045
Nombre d'arcs	607727
Nombre de relations is a	60096
Nombre de relations part of	6595
Total GeneProducts	65691
Total GeneProducts non annotés	0
Total GOTerms	39354
Total Annotations	541036

Le graphe contient environ 65 000 sommets correspondant à des GeneProducts. Il ne s'agit pas de gènes au sens strict, mais de GeneProducts, tels que des protéines ou des isoformes, tels qu'ils sont définis dans les fichiers d'annotation GOA. Un même gène pouvant produire plusieurs GeneProducts, ce nombre est cohérent avec les estimations du nombre de gènes humains, de l'ordre de 20 000 gènes codants.

La totalité GeneProducts de Homo sapiens possèdent au moins une annotation GO, ce qui témoigne de la qualité et de la richesse de l'annotation pour cet organisme.

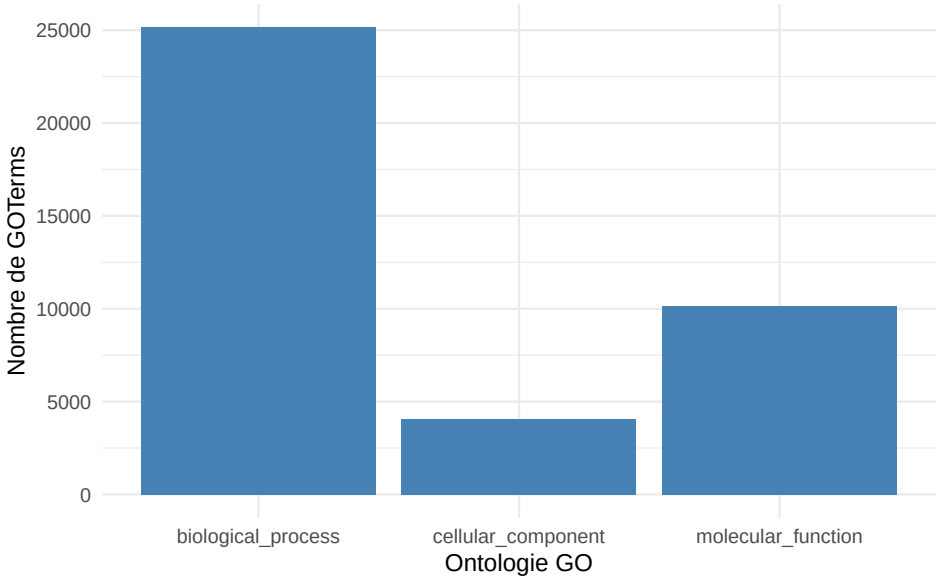
4.2 Répartition des GOTerms par ontologie.

Dans la figure 2, la répartition des GOTerms par ontologie est representée. Nous pouvons observer qu'environ 25000 GOTerms soit 64% des GOTerms sont liés a des processus biologiques contre 26% et 10% liés respectivement à de fonctions moléculaires et composants cellulaires.

Cette distribution reflète la structure intrinsèque de la Gene Ontology, dans laquelle l'ontologie Biological Process est la plus riche et la plus détaillée.L'ontologie Biological Process regroupe les processus dynamiques qui se déroulent dans le temps, comme les voies métaboliques, la régulation de l'expression génique entre autres. Ces processus sont complexes, très fortement hiérarchisés et souvent décomposables en sous-processus.

À l'inverse, les ontologies Molecular Function et Cellular Component reposent sur des concepts plus stables ce qui explique un nombre plus limité de termes. Cette répartition est cohérente avec les observations généralement rapportées pour la Gene Ontology complète.

Figure 2 : Repartition des GOTerms par ontologie



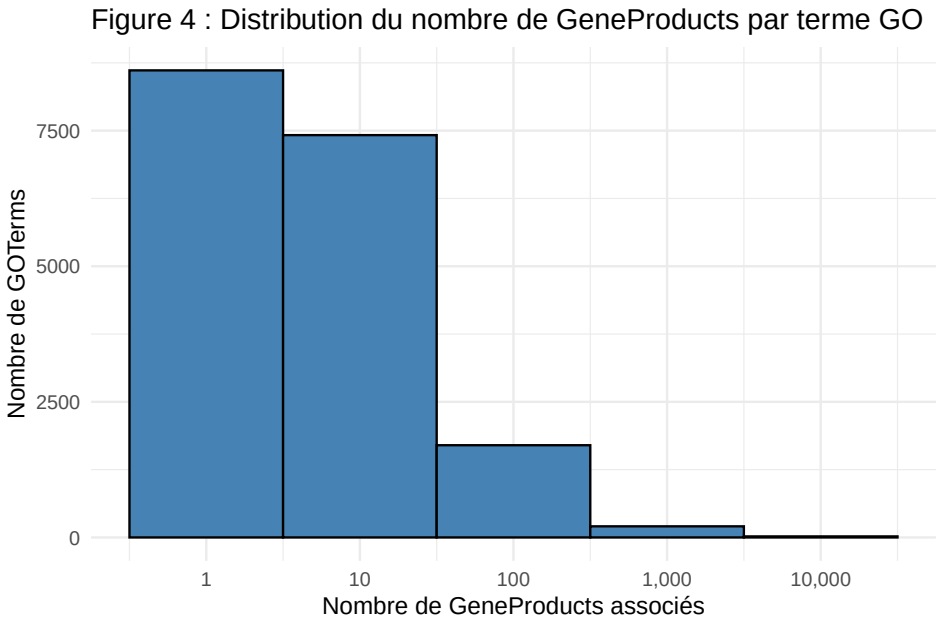
4.3 Profondeur maximale par ontologie

Table 3: Figure 3 : Plus long chemin par ontologie.

Ontologies	Max_depth
biological_process	18
molecular_function	13
cellular_component	14

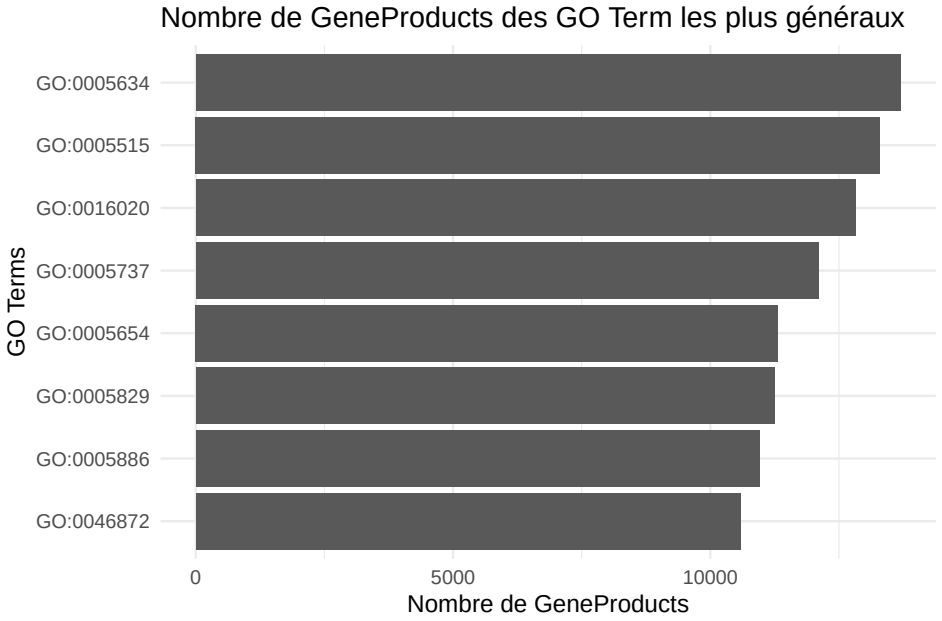
La profondeur maximale varie selon les ontologies GO. Biological Process présente la hiérarchie la plus profonde (18), ce qui montre la complexité et la spécialisation des processus biologiques. Cellular Component montre une profondeur intermédiaire (14), correspondant à une organisation spatiale de la cellule. Molecular Function est la moins profonde (13), ce qui est cohérent avec le caractère des fonctions moléculaires.

4.4 Distribution du nombre de GeneProducts par GOTerm.



La Figure 4 présente la distribution du nombre de GeneProducts associés à chaque terme GO. On observe une distribution fortement asymétrique : la majorité des GOTerms sont associés à très peu de GeneProducts, voire à aucun, tandis qu'un nombre restreint de termes très généraux concentre un grand nombre d'annotations.

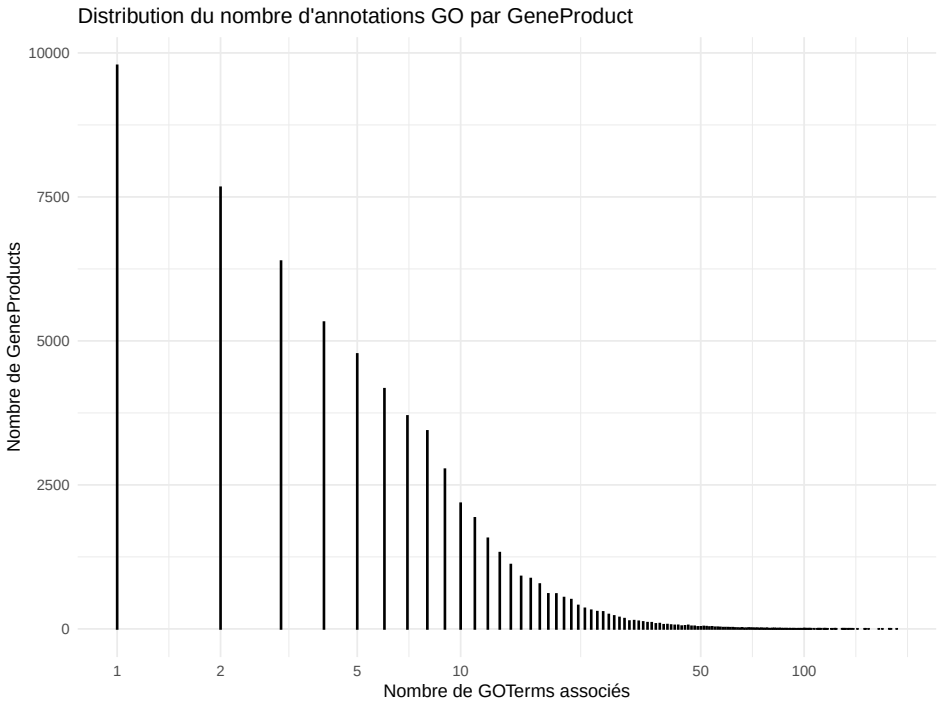
Cette distribution reflète la structure hiérarchique de la Gene Ontology, dans laquelle de nombreux termes correspondent à des concepts biologiques très spécifiques, rarement utilisés comme cibles d'annotation directe, alors que certains termes plus généraux agrègent un grand nombre de GeneProducts via la propagation hiérarchique des annotations. La répartition correspond a un arbre où les termes les plus généraux (tout en haut) ont une association avec un plus grand nombre de GeneProducts.



Nous pouvons observer que quelques GOTerms, qui sont plus généraux, sont rélies à des nombreux GeneProducts. Ainsi, un GeneProduct annoté avec un terme spécifique est également implicitement associé à l'ensemble des ancêtres de ce terme, ce qui conduit à une accumulation importante de GeneProducts pour les termes les plus généraux.

4.5 Distribution du nombre d'annotations par GeneProduct

Nous cherchons à déterminer le nombre de GOTerms associés à chaque GeneProduct afin d'identifier les gènes spécialisés et les gènes multifonctionnels.



L'analyse de la distribution du nombre d'annotations GO par GeneProduct révèle une organisation très déséquilibrée. La plupart des GeneProducts présentent peu d'annotations, suggérant des rôles fonctionnels plutôt ciblés. À l'opposé, un nombre limité de GeneProducts est associé à de nombreux termes GO, indiquant une implication transversale dans plusieurs processus biologiques. Cette structure est cohérente avec la présence de protéines centrales jouant un rôle clé dans les réseaux biologiques humains.

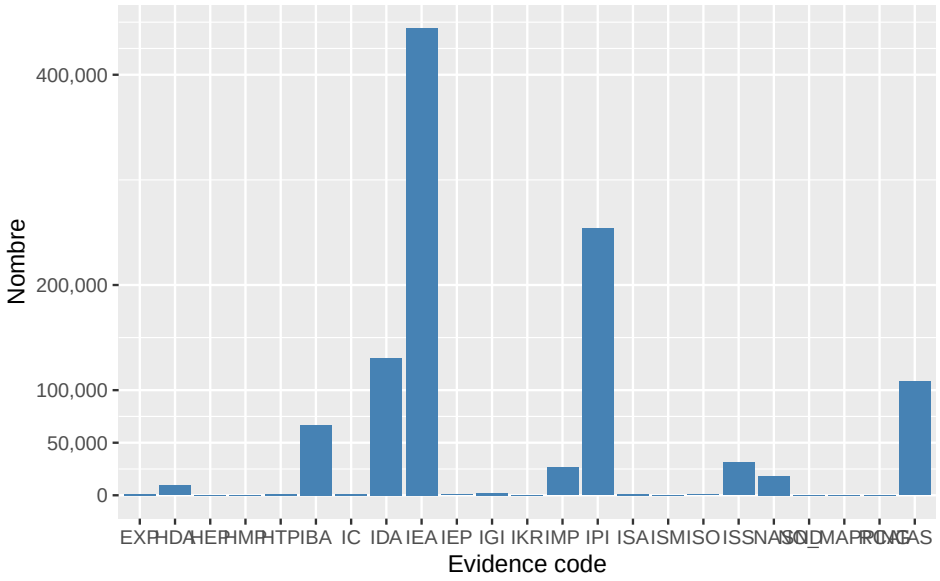
4.6 Répartition des evidence codes.

La Figure 5 montre la distribution des evidence codes associés aux annotations GO pour Homo sapiens. On constate que certains codes sont largement majoritaires, notamment ceux correspondant à des annotations automatiques ou issues de bases de données comme IEA, tandis que les annotations fondées sur des preuves expérimentales directes sont moins nombreuses (EXP,IDA,IMP).

Cette répartition est attendue pour un organisme bien étudié et largement annoté : les méthodes automatiques permettent de couvrir un grand nombre de gènes, tandis que les annotations expérimentales

apportent une information de haute qualité mais plus coûteuse à produire. Cette observation souligne l'importance de considérer les evidence codes lors de l'interprétation biologique des annotations GO.

Figure 5 : Distribution des evidence codes dans les GO.



5 Discussion

Les résultats obtenus nous montrent comment le traitement de graphes peut nous aider à mieux comprendre la Gene Ontology et les annotations associées à Homo sapiens. La taille du graphe reconstruit, tant en nombre de sommets qu'en nombre d'arêtes, illustre la complexité structurelle de la Gene Ontology.

L'analyse des distributions d'annotations met en évidence une forte hétérogénéité dans le nombre de GOTerms associés aux GeneProducts. Cette observation est biologiquement cohérente car certains gènes jouent un rôle central et participent à de nombreux processus biologiques, alors que d'autres ont des fonctions plus spécialisées et uniques.

La répartition des evidence codes souligne également un point important, une grande partie des annotations repose sur des méthodes automatiques ou des inférences issues de bases de données existantes, tandis que les annotations fondées sur des preuves expérimentales sont proportionnellement moins nombreuses. Ce constat rappelle que toutes les annotations GO n'ont pas le même niveau de fiabilité et que l'utilisation des evidence codes est essentielle pour une analyse biologique fiable et rigoureuse.

Du point de vue programmation, l'étude des complexités met en lumière certaines limites de l'approche naïve, mais que nous avons essayé d'optimiser.

Ces observations permettent d'évaluer dans quelle mesure les besoins initiaux du projet ont été satisfaits et ouvrent la voie à des pistes d'amélioration et de prolongement.

6 Bilan et perspective

Nous sommes globalement satisfaits des résultats de ce projet. La Gene Ontology et les annotations associées à Homo sapiens ont été correctement chargées et representées sous la forme d'un graphe orienté acyclique. Les relations hiérarchiques (is_a, part_of) ainsi que les relations d'annotation entre GeneProducts et GOTerms ont été intégrées de manière cohérente et exploitable.

Les principales fonctionnalités attendues ont été implémentées : navigation dans la hiérarchie GO, récupération des termes associés à un GeneProduct et inversement, calcul de la profondeur maximale de l'ontologie, ainsi que l'extraction de statistiques biologiques pertinentes. Les tests unitaires mis en place permettent de valider le bon fonctionnement des fonctions principales et renforcent la robustesse de l'implémentation.

Les outils développés dans ce projet ouvrent la voie à plusieurs types d'analyses biologiques comme des études fonctionnelles globales d'un génome à partir de la distribution des annotations GO, des comparaison de profils fonctionnels entre différents organismes, identification de gènes multifonctionnels ou fortement impliqués dans certains processus biologiques d'intérêt et bien sûr évaluer la qualité de l'annotation d'un organisme.

Plusieurs axes d'amélioration peuvent être envisagés :

- Optimisation algorithmique
- Extension des fonctionnalités
- Scalabilité (possibilité de traiter plusieurs organismes simultanément)

7 Section spéciale méthodologie de travail

Ressources utilisées:

- StackOverflow
- Claude Haiku 4.5 (Utilisation pour rédaction, corriger les docstrings en suivant PEP 257, idées d'optimisation de code pour les premieres implementations, (notamment fonctions d'indexation **GeneProducts**) implementation faite a la main sans IA)
- Cours Traitement de Graphes et réseaux biologiques.
- Discussion entre groupe de classe notamment sur les 3 principales fonctions de geneontology.py
- https://peps.python.org/
- https://ggplot2.tidyverse.org

8 Annexes

Organisation détaillé du projet

{

```
## ..
## +-- LICENSE
## +-- README.md
## +-- data
## |   +-- 25.H_sapiens.goa
## |   +-- README.md
## |   +-- dressing.tsv
## |   +-- edges.csv
## |   +-- go-basic.obo
## |   \-- nodes.csv
## +-- figures
## |   +-- Distribution_du_nombre_de_GeneProducts_par_terme_GO.png
## |   +-- Distribution_of_evidence_codes_in_GO_annotations.png
## |   +-- Nombre_de_GeneProducts_des_GO_Term_les_plus_generaux.png
## |   +-- Number_per_evidencecode.png
## |   +-- repartition_goterms_ontologie.png
## |   \-- tree.png
## +-- output
## |   +-- evidence_code_distribution.csv
## |   +-- geneproducts_per_goterms.csv
## |   +-- goterms_by_namespace.csv
## |   +-- goterms_per_geneproduct.csv
## |   +-- max_depth_per_ontology.csv
## |   +-- number_of_geneproducts_per_number_of_annotations.csv
## |   +-- tests_summary.csv
## |   \-- tests_time.csv
## +-- report
## |   +-- Projet_GeneOntology.Rmd
## |   +-- Projet_GeneOntology.pdf
## |   \-- Projet_GeneOntology_files
## |       \-- figure-latex
## |           +-- unnamed-chunk-4-1.pdf
## |           +-- unnamed-chunk-6-1.pdf
## |           +-- unnamed-chunk-7-1.pdf
## |           +-- unnamed-chunk-8-1.pdf
## |           \-- unnamed-chunk-9-1.pdf
## +-- src
## |   +-- __init__.py
## |   +-- __pycache__
## |   |   +-- __init__.cpython-314.pyc
## |   |   +-- geneontology.cpython-314.pyc
## |   |   \-- graphmaster.cpython-314.pyc
## |   +-- geneontology.py
```

```
## |   \-- graphmaster.py
## +-- stats.py
## +-- test_OBO_GOA.py
## +-- test_geneontology.py
## +-- test_graphmaster.py
## +-- tests
## |   +-- __init__.py
## |   \-- times_tests.csv
## \-- tests_time.py
```

}

Dans ce projet, seules les relations hiérarchiques is_a et part_of ont été prises en compte pour la navigation et le calcul de la profondeur. Les autres relations présentes dans la Gene Ontology (regulates, positively_regulates, has_part, etc.) décrivent des liens fonctionnels ou contextuels et n'ont pas été utilisées afin de préserver une définition stricte de la hiérarchie.

Les fonctions principales ont été validées à l'aide de tests unitaires basés sur des graphes jouets construits manuellement, permettant de vérifier le comportement attendu des algorithmes. Un test supplementaire a été réalisé sur le graphe complet de la Gene Ontology et les annotations de Homo sapiens, en comparant manuellement les résultats obtenus avec des informations issues de sources publiques (UniProt, AmiGO).